HTTP request level. The best way to do HTTP mocking in Elixir is with the Bypass library, so let's add that to our project.

Bypass[1] offers a pretty amazing testing capability. Because Elixir is able to spin up multiple applications at once, Bypass takes advantage of this and spins up actual HTTP servers. Instead of mocking the HTTP request at the source, it mocks HTTP requests at the destination! This approach is useful because it tests that your entire HTTP stack works as expected.

Add the Bypass library in your mix.exs file:

phone_app/mix.exs

```
{:bypass, "~> 2.1"}
```

And then run mix deps.get to pull in the dependency. In addition, add the following code to test.exs:

phone_app/config/test.exs

```
config :phone_app, :twilio,
  key_sid: "mock-key-sid",
  key_secret: "mock-key",
  account_sid: "mock-account",
  number: "+19998887777",
  base_url: "http://localhost:4005/2010-04-01"
```

And also create the following JSON file at test/support/fixtures/success.json:

phone_app/test/support/fixtures/success.json

```
{
  "account_sid": "account_sid",
  "sid": "sid",
  "body": "body",
  "from": "+11112223333",
  "to": "+19998887777",
  "status": "sent"
}
```

Now, we can write a test for our API requests. We'll do this by testing the Conversations.send_sms_message/1 function. This test will ensure that our HTTP request works and that it's handled correctly by the send_sms_message/1 function.

Create test/phone_app/conversations/conversations_test.exs and add the following code:

phone_app/test/phone_app/conversations/conversations_test.exs

```
Line 1 defmodule PhoneApp.ConversationsTest do
  -   use PhoneApp.DataCase, async: true
  -
  -   alias PhoneApp.Conversations
  5   alias PhoneApp.Conversations.Schema.NewMessage
```

1. https://hexdocs.pm/bypass/Bypass.html